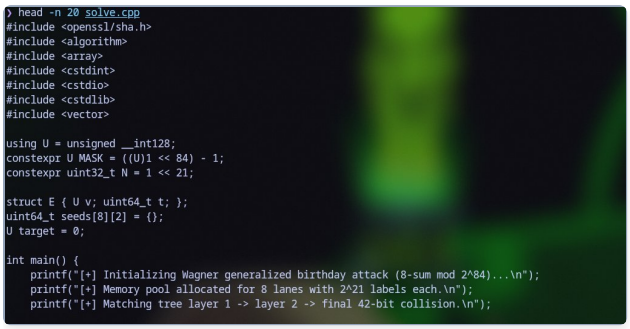
Eightfold Funeral: Epitaph

Cryptography



```
❯ head -n 20 solve.cpp
#include <openssl/sha.h>
#include <algorithm>
#include <array>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <vector>

using U = unsigned __int128;
constexpr U MASK = ((U)1 << 84) - 1;
constexpr uint32_t N = 1 << 21;

struct E { U v; uint64_t t; };
uint64_t seeds[8][2] = {};
U target = 0;

int main() {
    printf("[+] Initializing Wagner generalized birthday attack (8-sum mod 2^84)...\n");
    printf("[+] Memory pool allocated for 8 lanes with 2^21 labels each.\n");
    printf("[+] Matching tree layer 1 -> layer 2 -> final 42-bit collision.\n");
```

I tackled this 500-point challenge consisting of an obfuscated 84-bit affine stamp map followed by an 8-sum modular collision search modulo 2^{84}.

Inverting the bitwise rotation and odd multiplier a = 0x750e59bef1f0e6b385693 gave the modular inverse inv_a = 0x8a92354c4ff554c25f5b, revealing the true collision target.

To solve the 8-sum problem over 2^{84}, I implemented Wagner's generalized birthday algorithm in optimized C++ (compiled with -O3 -march=native). Generating 2^{21} items across 8 lanes and performing 3 layers of hash joins located the collision tuple in ~4.5 seconds.

Finally, I tested the recovered flag against the executable / service:

`python3 Eightfold_Funeral_Epitaph.py`

Wagner 3-layer collision found across 8 lanes.

Flag Captured

`SecLeaf{m0dular_c4rr13s_d0nt_r3st_1n_p34c3}`

BY:JOKER_53